MenuNorm: Two-Level Standardization of Noisy Korean Product Names with Dual-Segmentation Fusion and Guaranteed Category Coverage

Minyoung Park
Department of Computer Science, Gyeongsang National University · Dream2Real
github.com/pmy02/Menunorm

Abstract. User-generated Korean product names are noisy: decorations (★ 짜장면 ★), option suffixes (세트, 곱빼기), spacing errors, Korean–English code-switching (cheese 돈가스), and spelling variants inflate a small menu vocabulary into millions of surface forms. We present MenuNorm, a reproducible pipeline that standardizes such names at two levels: a fine-grained canonical label (136 classes) assigned with an abstention option, and a coarse delivery-app category (13 classes) assigned with guaranteed full coverage through a fallback chain. The matcher fuses two segmentations—a morphological path (MeCab with an auto-compiled domain user dictionary) and a surface path—by taking the per-row maximum of character n-gram TF–IDF scores, which we show dominates either path alone. Because the motivating industrial dataset (∼20M records) is under NDA, we release a deterministic synthetic benchmark whose noise model reproduces the documented real-world corruption modes, whose class mix is calibrated to published demand statistics, and whose hard track injects character-level typos covered by no lexicon. The benchmark also plants a deliberately noisy store-category column to block shortcut solutions. On the hard track, MenuNorm reaches 98.4 % label accuracy and 99.7 % category accuracy at 100 % category coverage; on the 224 rows where the label matcher abstains, the category remains correct 85.3 % of the time, quantifying graceful degradation across levels. All results regenerate from a seed in about one second on a single core. Code, lexicons, benchmark generator, and evaluation harness are public.

Keywords: text normalization, entity matching, Korean NLP, product categorization, selective prediction, synthetic benchmarks

1 Introduction

Commerce platforms ingest product names typed by thousands of independent merchants. In Korean food delivery, the same item surfaces as 짜장면, ★ 짜장면 ★ 500ml, 짜장면곱빼기, or [강남점] 짜장면세트: promotional decorations, option and quantity suffixes, store-branch prefixes, spacing errors, spelling variants, and Korean–English code-switching multiply a vocabulary of a few hundred menu items into millions of distinct strings. Search, analytics, and recommendation all degrade on this long tail.

This work grew out of a 2023 industry collaboration in which roughly 20M raw product records were standardized into about one thousand canonical labels. Two problems survived that engagement. First, the original workflow was methodologically fragile: normalization rules were edited by hand inside notebooks, one regular expression at a time, with progress tracked by re-saving a spreadsheet under incremented version names. Second, the data is under NDA, so none of the reported outcomes were externally verifiable. MenuNorm is a systematic rebuild that addresses both: the method becomes a configuration-driven, tested, auditable pipeline, and evaluation moves to an open synthetic benchmark on which every number in this paper is measured and reproducible from a fixed seed.

A further observation shapes the design. Downstream consumers rarely need only the finest label. Delivery platforms organize browsing around 10–20 coarse categories (치킨, 한식, 중식, ...), and business reporting is published at that granularity. Yet a fine-grained matcher that honestly abstains on ambiguous inputs leaves those rows without any assignment. We therefore formulate standardization at **two levels**: level 2 assigns one of 136 canonical labels with an abstention option, while level 1 assigns one of 13 delivery-app categories with guaranteed 100 % coverage via a fallback chain—and we measure, rather than assume, how well the coarse decision survives when the fine decision refuses.

Contributions.

1. A two-level formulation of product-name standardization that pairs selective fine-label pre-

diction with total coarse-category assignment, and a metric—category accuracy on abstained rows—that quantifies graceful degradation between the levels (§ 3).
2. A delivery-app category taxonomy whose 13 classes, exclusions (e.g., time-based 야식), and single-assignment policy are grounded in platform documentation and published demand statistics, with a public decision log (§ 4).
3. A dual-segmentation matcher that fuses a morphological path (MeCab plus an auto-compiled user dictionary) and a surface path by per-row maximum of character n-gram TF–IDF scores; an ablation shows fusion dominates both single paths (§ 5, § 7).
4. An open, deterministic, two-track benchmark: the standard track reproduces documented real-world noise modes; the hard track adds character-level typos covered by no lexicon; the class mix is calibrated to published demand shares; and a deliberately noisy store-category column blocks shortcut learning (§ 6).

2 Related Work

Entity matching and name normalization. Mapping noisy surface strings to canonical entities is the classical data matching problem [5]. String-similarity approaches remain strong baselines for name variants [6], and character n-grams in particular are robust to segmentation and spelling noise [1, 12], which motivates our matcher. Large-scale product categorization in e-commerce has mostly been studied as supervised classification over catalog taxonomies [2]; our setting differs in that no labeled training corpus exists on the open side of the NDA boundary, pushing us toward a lexicon-plus-similarity design whose behavior is fully inspectable.

Korean morphological analysis. Agglutinative Korean benefits from morphological segmentation; MeCab's CRF formulation [10] with the mecab-ko-dic dictionary is the standard open-source stack, accessible from Python ecosystems such as KoNLPy [13]. Stock dictionaries, however, over-segment food-domain loanwords (아인슈페너 → 아인 + 슈페너), an error class we repair with an automatically compiled user dictionary and quantify in § 7.

Selective prediction. Allowing a classifier to abstain below a confidence threshold trades coverage for precision [3, 7]. We apply the reject option at the fine level only, and study the previously unreported interaction with a coarser level that must not abstain.

Synthetic benchmarks and shortcuts. When real data cannot be released, generated data with a controlled error model is an established evaluation vehicle in record linkage [4]. We follow that tradition and additionally guard the benchmark against shortcut learning [8] by injecting label noise into the one metadata column (store category) that would otherwise let a model bypass the text entirely.

3 Task Formulation

Let $\mathcal{L}$ be a set of canonical labels ($|\mathcal{L}|$=136) and $\mathcal{C}$ a set of categories ($|\mathcal{C}|$=13) with a surjective map $\kappa : \mathcal{L} \to \mathcal{C}$. Given a raw name x (plus store metadata), the system outputs

$$\hat{\ell}(x) \in \mathcal{L} \cup \{\texttt{UNMATCHED}\}, \qquad \hat{c}(x) \in \mathcal{C},$$

i.e., the fine label may abstain but the category may not.

Level-2 metrics follow selective prediction: coverage $\phi = \Pr[\hat{\ell} \neq \texttt{UNMATCHED}]$, matched accuracy $\Pr[\hat{\ell} = \ell \mid \hat{\ell} \neq \texttt{UNMATCHED}]$, and overall accuracy $\Pr[\hat{\ell} = \ell]$, where abstentions count as errors.

Level-1 metrics are category accuracy $\Pr[\hat{c} = c]$ and category coverage (fraction of rows assigned; 1.0 by construction, verified at run time). We additionally report

$$A_{\text{unm}} \;=\; \Pr\Big[\hat{c} = c \,\Big|\, \hat{\ell} = \texttt{UNMATCHED}\Big],$$

the category accuracy on abstained rows. A_{unm} measures graceful degradation: when the evidence is too weak for a fine decision, how often is the coarse decision still right? A pipeline could trivially reach 100 % category coverage by guessing; A_{unm} makes the quality of the forced assignments visible.

A stage-0 filter also removes out-of-scope rows (packaged retail goods, non-food stores); we report its leakage separately.

4 Category Taxonomy

The 13 categories mirror the browsing rail of Korean delivery platforms: 치킨, 한식, 중식, 카페·디저트, 분식, 찜·탕·찌개, 돈까스·회·일식, 피자, 버거·패스트푸드, 족발·보쌈, 고기·구이, 양식, 아시안. Five principles govern the design.

1. **Platform-aligned.** Names and boundaries follow the taxonomies merchants and consumers already use, so outputs are comparable to industry reporting.
2. **Food-type only.** Platforms mix food categories with functional ones; Yogiyo's partner documentation defines the late-night category (야식) purely by store operating hours [15]. Time- and function-based groupings are

not properties of a product name and are excluded.

3. **A partition, not tags.** Platforms surface one store in up to three categories [15]; analytics needs mutually exclusive, exhaustive classes, so exactly one category is assigned per row.
4. **10–20 classes,** matching the visible category rail and keeping per-class support meaningful.
5. **Share-calibrated benchmark.** The synthetic benchmark samples categories by a `benchmark_share` weight informed by published demand analyses—card-payment studies place chicken first [9], and consumer surveys consistently rank chicken, Chinese food, pizza, and jokbal/bossam at the top—yielding a chicken-heavy (0.18) distribution with a thin Asian tail (0.01). We state explicitly that these weights are a design choice informed by heterogeneous sources, not a market measurement.

Boundary decisions are recorded in a public log: raw-fish labels fold into 돈까스·회·일식 following platform compound naming; 마라탕 sits under 중식 as platforms list it; sandwiches and bagels go to 카페·디저트 by dominant store type while toast and hot dogs go to 버거·패스트푸드. Categories without label coverage (도시락·죽) are deferred rather than left empty.

Assignment with guaranteed coverage. Level-1 assignment follows a fallback chain: (i) the category $\kappa(\ell^*)$ of the highest-scoring canonical label ℓ^*, used even when the label matcher abstains—the coarse decision can survive fuzzy evidence that the fine decision rightly refuses; (ii) if the name is empty after normalization, the store's own category when it is a known food category; (iii) otherwise the batch majority. Every row records which branch fired, keeping the mechanism auditable, and A_{unm} evaluates branch (i) on exactly the rows where it matters.

5 The MenuNorm Pipeline

Stage 0: target filtering. Rows from non-food stores are dropped by store category; rows carrying a syntactically valid EAN-13 barcode (check-digit verified) are dropped as packaged retail goods.

Stage 1: rule normalization. An ordered, declarative rule set replaces the original hand-edited regex workflow: Unicode NFKC folding; an offline English→Korean glossary (with space collapsing inside Latin runs, so ch oco 라떼 still translates); bracketed-content and decoration removal; option/quantity patterns (glued suffixes such as -세트, promotions such as 1+1, branch prefixes); a spelling-variant map that also matches space-collapsed forms; residual digit/Latin cleanup. Each row logs which stages fired (`rules_fired`), so every transformation is reviewable.

Stage 2: dual segmentation. Two token sequences are produced per name: a morphological path (MeCab [10] with a user dictionary compiled automatically from a plain word list; final-consonant features derived arithmetically from Unicode) and a dependency-free surface path (character-class chunks). Stopword removal is span-aware: a stopword matches a single token or the concatenation of up to three adjacent morphemes, and runs before the part-of-speech filter—otherwise MeCab's copula tag inside 시그니처 (→ 시그/니/처) would delete the middle morpheme and the stopword could never re-join.

Stage 3: canonicalization with score fusion. Both candidate strings are scored against $\mathcal{L}$ by exact match after space removal, else by cosine similarity over character 2–4-gram TF–IDF [1] (spaced and space-collapsed query forms, maximum taken). The per-row fused score is the maximum over the two paths, ties preferring the surface candidate, which never loses characters. Scores below a threshold τ abstain. The rationale is complementary brittleness: morphological analysis loses characters under spacing noise (돼지국밥→ 국밥; 쫄면 tagged as a verb form and deleted), while surface matching is weaker on glued compounds; fusion keeps the stronger signal per row, and § 7 quantifies the claim.

Stage 4: category assignment implements the fallback chain of § 4.

Stage 5: evaluation and discovery. Beyond the metrics of § 3, a threshold sweep is emitted from the single scoring pass, and abstained names are clustered (k-means [11] over the same TF–IDF space) so that each cluster's most frequent member becomes a proposal for a new canonical label—turning residual mass into a human review queue instead of silent errors.

The implementation is a tested Python package (scikit-learn matcher backend [14]; 35 unit and end-to-end tests; continuous integration over Python 3.10/3.12 and both tokenizer backends, so the pipeline stays runnable without MeCab).

6 An Open Two-Track Benchmark

Following the synthetic-data tradition of record linkage evaluation [4], we release a deterministic generator instead of the NDA data.

Noise model. Each sampled canonical name receives independent corruption operations

Table 1: Main results (20k rows, seed 42). Level-2 = canonical label with abstention; Level-1 = category with guaranteed coverage.

	Level-2 (label)			Level-1
Track	Cov.	Match.	Overall	Cat. acc.
standard	100.0	100.0	**100.0**	**100.0**
hard	98.75	99.68	**98.44**	**99.72**

mirroring the documented real-world modes: decorations and promo tags (★...★, [인 기]); option and quantity suffixes (세 트/셋 트/단 품, 1+1, 2 인분, 500ml); temperature prefixes for beverages (ICE/아이스); random space insertion; Korean→English component swaps invertible by the glossary (치즈 →cheese); spelling variants (돈가스 → 돈까스); store-branch prefixes; and, mixed in, packaged retail rows with valid EAN-13 codes plus rows from non-food stores (gold label `NOT_TARGET`) that the stage-0 filter must remove.

Two-level sampling. A category is drawn by `benchmark_share` (§ 4); a label inside it follows a Zipf distribution (s=1.05) [16], reproducing the long-tail structure that motivated the original project. The generated mix tracks the target within ±0.005 per category.

Anti-shortcut design. With probability 0.15 a row's store category differs from its item's category (a bunsik store selling 돈가스), so predicting the category by copying the store column is penalized; the pipeline classifies from text and uses the store column only inside fallback branch (ii) [8].

Tracks. The standard track uses only the modes above, all of which the lexicons cover—so it functions as a closed-world regression test whose ceiling is 100 %. The hard track additionally applies, with probability 0.12, one character-level typo (deletion, adjacent transposition, or duplication) that no lexicon covers, probing genuine fuzzy-matching robustness. Both tracks: 20,000 rows, seed 42; 6,912 (standard) and 7,981 (hard) unique raw strings collapse onto the 136 labels.

7 Experiments

Setup. Threshold τ=0.45 (chosen from the sweep, Fig. 1), character n-grams $n \in [2, 4]$, MeCab with the compiled user dictionary as the morphological path. Metrics are computed over target rows; the stage-0 filter removed 1,889 (standard) and 2,024 (hard) contaminant rows with zero leakage on both tracks.

Main results (Table 1). The perfect standard-track score should be read as designed: it certifies that the pipeline fully inverts every known noise mode—a closed-world regression property, not a real-world claim. The hard track is the informative one: 98.44 % label accuracy and 99.72 % category accuracy at 100 % category coverage under typos unseen by any lexicon. Category confusion is near-diagonal (Fig. 2).

Table 2: Segmentation ablation, hard track (%).

Path	Coverage	Matched	Overall
Morphological only	93.14	99.30	92.48
Surface only	98.58	99.76	98.34
Fused (max score)	**98.75**	99.68	**98.44**

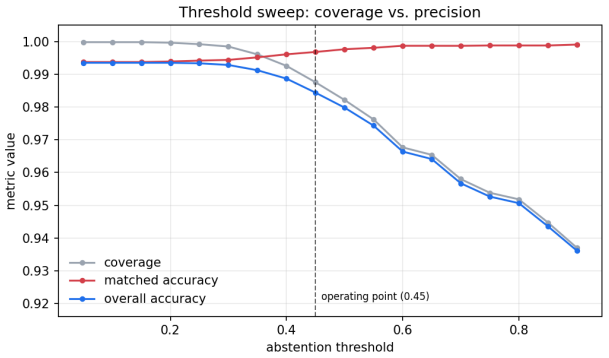


Figure 1: Coverage/precision trade-off across the abstention threshold (hard track); the operating point τ=0.45 is chosen from this sweep, which the pipeline re-emits on every run.

Fusion ablation (Table 2). Each path fails where predicted: the morphological path loses characters under spacing noise; the surface path lacks segmentation on glued compounds. Per-row max fusion dominates both on coverage and overall accuracy—and, usefully, the decision is data-driven per row rather than a global tokenizer choice.

Graceful degradation. On the 224 hard-track rows where the label matcher abstained, the fallback-assigned category was correct A_{unm}=85.3 % of the time. Example: ♥ 부찌대개세트 ♥ (a doubly-transposed 부 대 찌 개 세 트) scores only 0.21 against its best label and is rightly refused at level 2, yet its category resolves correctly to 찜 · 탕 · 찌 개. The coarse decision survives evidence the fine decision cannot commit to.

User dictionary. The stock dictionary over-segments food loanwords; the compiled user dictionary restores single tokens, e.g. 아인슈페너 (아인/슈페너 → 아인슈페너), 크로플 (크로/플 → 크로플), 마라샹궈 (마라/샹/궈 → 마라샹궈), 알리오올리오 (알/리오/올리/오 → 알리오올리오). This lifts the morphological path and explains part of its remaining value inside the fusion.

Threshold behavior (Fig. 1). Matched accu-

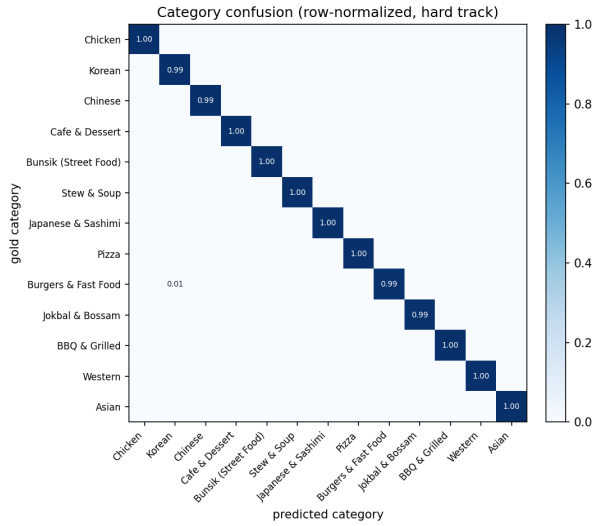


Figure 2: Row-normalized category confusion on the hard track.

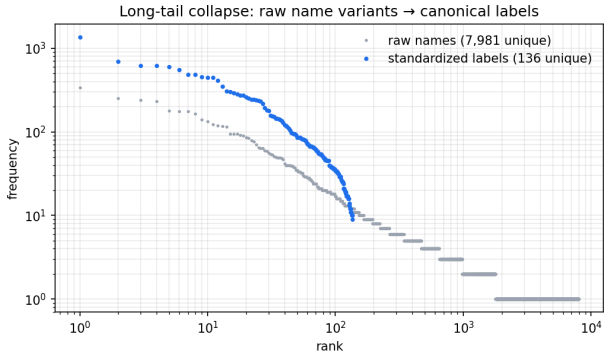


Figure 3: Rank–frequency view of the long-tail collapse: 7,981 unique raw strings map onto 136 canonical labels (hard track).

racy rises and coverage falls monotonically with τ, the classical reject-option trade-off [3, 7]; the sweep costs no extra similarity computation because scores are threshold-independent.

Efficiency and reproducibility. A full 20k-row run completes in roughly one second on a single core (Python 3.12), including user-dictionary compilation. Given the seed, generation and evaluation are deterministic; every table and figure in this paper regenerates from three commands.

Error analysis. Residual hard-track label errors concentrate in (i) typos inside Latin runs that defeat the glossary after corruption and (ii) deletions that make two labels nearly equidistant (크로와상 drifting toward 크로플); the abstention mechanism converts a majority of the latter into `UNMATCHED` plus a correct category rather than a confident wrong label.

8 Limitations

The benchmark is synthetic: although its noise model is drawn from documented real-world modes and the hard track removes lexicon coverage as a confound, absolute numbers remain optimistic relative to open-world production data (unknown typo classes, OCR noise, emerging menu items). The `benchmark_share` weights are a design choice informed by heterogeneous published statistics, not a market measurement, and should not be cited as such. Code-switching recovery depends on an offline glossary; unseen romanizations fall through to `UNMATCHED` (with a category still assigned) rather than being guessed. Finally, the canonical inventory (136 labels) is deliberately small; scaling to 10^3 labels with approximate-nearest-neighbor search, and adding an embedding-based third fusion candidate under the same ablation protocol, are the natural next steps. The original engagement's reported outcome (∼20M records to ∼1K labels at ∼95 % on a hand-audited sample) is provided as motivating context only and is not reproducible from the released artifacts.

9 Conclusion

MenuNorm recasts an ad-hoc industrial standardization effort as a reproducible research artifact: a two-level formulation that reconciles honest abstention with total category coverage, a fusion matcher whose benefit is demonstrated by ablation rather than asserted, a taxonomy whose every boundary decision is sourced and logged, and a benchmark engineered to be hard to game—closed-world ceiling made explicit, shortcut column poisoned, typo track uncovered by any lexicon. We hope the release serves both as a practical tool for Korean commerce text and as a template for turning NDA-bound industrial work into verifiable science.

References

[1] W. B. Cavnar and J. M. Trenkle. N-gram-based text categorization. In Proc. 3rd Annual Symposium on Document Analysis and Information Retrieval (SDAIR), pages 161–175, 1994.

[2] A. Cevahir and K. Murakami. Large-scale multi-class and hierarchical product categorization for an e-commerce giant. In Proc. COLING, pages 525–535, 2016.

[3] C. K. Chow. On optimum recognition error and reject tradeoff. IEEE Transactions on Information Theory, 16(1):41–46, 1970.

[4] P. Christen. Probabilistic data generation for deduplication and data linkage. In Proc. IDEAL, pages 109–116. Springer, 2005.

[5] P. Christen. Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection. Springer, 2012.

[6] W. W. Cohen, P. Ravikumar, and S. E. Fienberg. A comparison of string distance metrics for name-matching tasks. In Proc. IJCAI Workshop on Information Integration on the Web (IIWeb), pages 73–78, 2003.

[7] R. El-Yaniv and Y. Wiener. On the foundations of noise-free selective classification. Journal of Machine Learning Research, 11:1605–1641, 2010.

[8] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann. Shortcut learning in deep neural networks. Nature Machine Intelligence, 2:665–673, 2020.

[9] Hana Institute of Finance. 배달앱이용현황과 메뉴유형별수요특성 (Delivery-app usage and demand characteristics by menu type). Card-payment data analysis report, 2022.

[10] T. Kudo, K. Yamamoto, and Y. Matsumoto. Applying conditional random fields to Japanese morphological analysis. In Proc. EMNLP, pages 230–237, 2004.

[11] S. P. Lloyd. Least squares quantization in PCM. IEEE Transactions on Information Theory, 28(2):129–137, 1982.

[12] P. McNamee and J. Mayfield. Character n-gram tokenization for European language text retrieval. Information Retrieval, 7(1–2):73–97, 2004.

[13] E. L. Park and S. Cho. KoNLPy: Korean natural language processing in Python. In Proc. 26th Annual Conference on Human and Cognitive Language Technology (HCLT), 2014.

[14] F. Pedregosa et al. Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12:2825–2830, 2011.

[15] Yogiyo partner documentation. 요기요카테고리 소개 (Introduction to Yogiyo categories). https://partner.yogiyo.co.kr, accessed 2026.

[16] G. K. Zipf. Human Behavior and the Principle of Least Effort. Addison-Wesley, 1949.